

BÁO CÁO LAB 7

TẤN CÔNG VÀ PHÒNG CHỐNG CROSS-SITE SCRIPTING (XSS)

Họ tên: UNG VẮNG ĐẠI LÂM MSSV: 2380601202 Lớp: 23DATA1

Hình thức: Cá nhân. Mã nguồn: github.com/UngVangDaiLam/ATw-CSDL_Lab (thư mục lab7).

⚠ Lưu ý: Thao tác trên app demo tự dựng (localhost), môi trường mô phỏng, không tác động hệ thống thật.

1. Môi trường thực hành

App demo **Node.js + Express** (`demo-app/`) mô phỏng trang tìm kiếm. Thay cho DVWA, app tự chứa lỗ hổng để thử nhanh. Hai trang để so sánh (Reflected XSS):

- **/search-vulnerable** — chèn thẳng từ khoá vào HTML → **dính Reflected XSS**.
- **/search-secure** — mã hoá (escape) đầu ra bằng hàm tương đương `htmlspecialchars()` → an toàn.

Chạy: `cd demo-app` → `npm install` → `npm start` → mở `http://localhost:7070`.

2. Khai thác lỗ hổng (Reflected XSS)

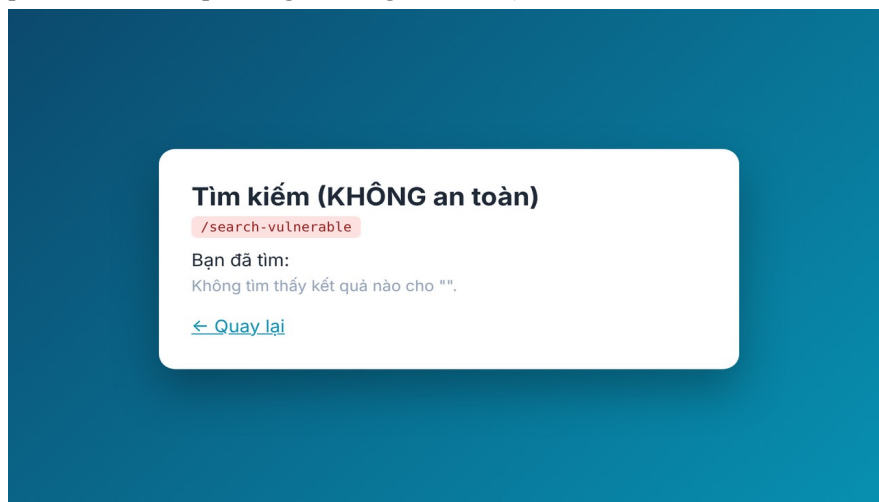
1. Vào trang tìm kiếm, nhập payload vào ô từ khoá:

```
<script>alert('XSS')</script>
<img src=x onerror=alert('XSS')>
```

2. Gửi tới **/search-vulnerable**. Server trả về trang HTML có chèn thẳng payload, nên trình duyệt coi `<script>` là mã và **thực thi**.

3. **Kết quả:** trình duyệt bật hộp thoại `alert("XSS")` → lỗ hổng tồn tại. Mình kiểm chứng tự động bằng Playwright (Hình minh hoạ: payload chèn banner đỏ chứng tỏ mã của kẻ tấn công đã chạy trong trang — Hình 2).

Trong thực tế, kẻ tấn công nhúng payload vào một **URL** rồi gửi link cho nạn nhân; khi nạn nhân mở link, mã chạy trong phiên của họ (đánh cắp cookie, chiếm phiên, giả mạo giao diện...).



Hình 2. Endpoint có lỗ hổng: mã chèn vào đã thực thi trong trình duyệt

3. Phân tích nguyên nhân

Nguyên nhân gốc: **dữ liệu người dùng được đưa thẳng ra HTML mà không mã hoá**. Ký tự `<`, `>` giữ nguyên nên trình duyệt hiểu `<script>...</script>` là một thẻ thực thi thay vì văn bản. Đoạn mã lỗi:

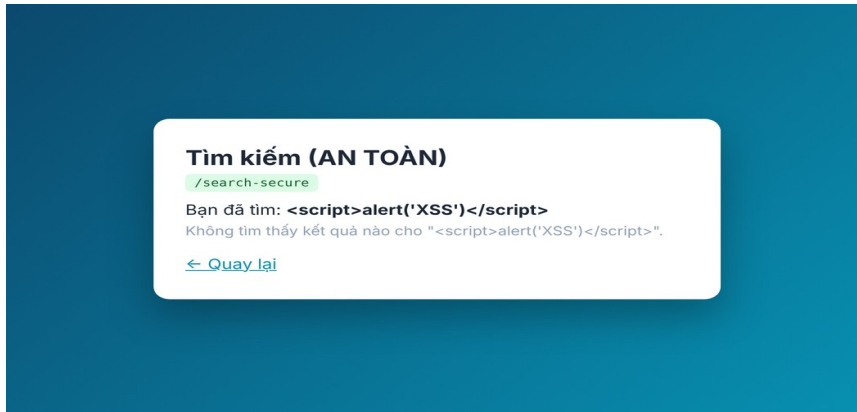
```
res.send('<p>Bạn đã tìm: <b>' + q + '</b></p>'); // q chèn thẳng -> XSS
```

4. Thực hiện phòng chống

Biện pháp chính: **mã hoá đầu ra (output escaping)** — đổi ký tự đặc biệt thành thực thể HTML trước khi in ra trang (giống `htmlspecialchars()` trong PHP):

```
function escapeHtml(s){ return s
  .replace(/&/g, '&amp;')
  .replace(/</g, '&lt;')
  .replace(/>/g, '&gt;')
  .replace(/"/g, '&quot;')
  .replace(/'/g, '&#39;'); }
res.send('<p>Bạn đã tìm: <b>' + escapeHtml(q) + '</b></p>');
```

Kiểm chứng: gửi lại payload `<script>alert('XSS')</script>` tới **/search-secure** → **không có alert**, payload hiển thị dưới dạng chữ (Hình 3). Lỗ hổng đã được vá.



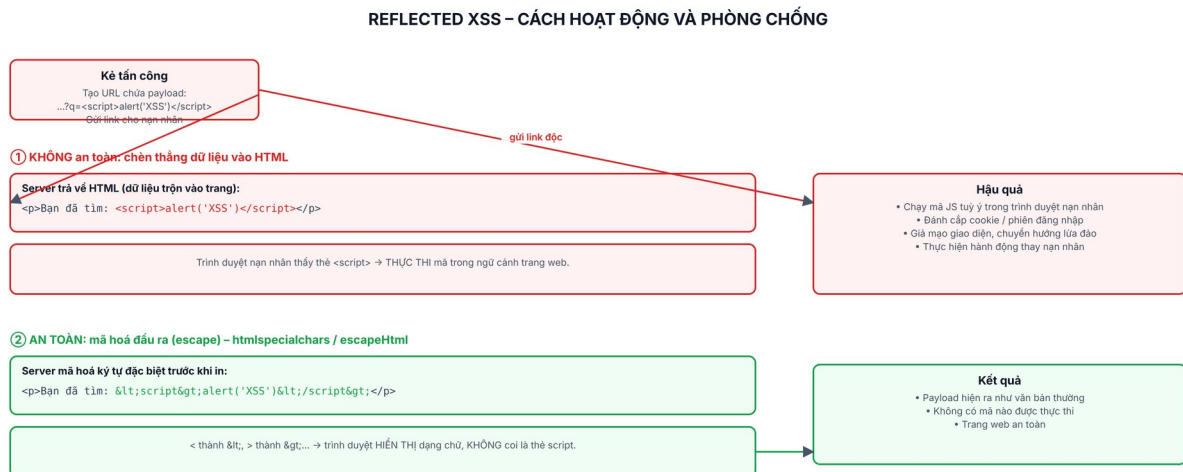
Hình 3. Endpoint an toàn: payload chỉ hiển thị dạng văn bản, không thực thi

5. Khuyến nghị phòng tránh XSS

- **Mã hoá đầu ra theo ngữ cảnh** (HTML, thuộc tính, JS, URL); dùng template engine tự escape (EJS, Handlebars, React...).
- **Kiểm tra & lọc đầu vào** (whitelist); với HTML do người dùng nhập, lọc bằng thư viện như **DOMPurify**.
- **Content Security Policy (CSP)** để chặn script lạ; đặt cookie **HttpOnly** để JS không đọc được cookie phiên.
- Tránh `innerHTML`, `document.write`, `eval` với dữ liệu người dùng; ưu tiên `textContent`.
- Dùng **security headers** (helmet), cập nhật framework, kiểm thử XSS định kỳ.

6. Kết luận

XSS xảy ra khi dữ liệu người dùng được đưa ra trang mà không mã hoá, cho phép chạy mã tùy ý trong trình duyệt nạn nhân. Thử nghiệm cho thấy một payload đơn giản đã thực thi trên trang có lỗ hổng, nhưng hoàn toàn vô hại sau khi áp dụng mã hoá đầu ra. Escaping đầu ra kết hợp CSP và cookie HttpOnly là các biện pháp phòng chống cốt lõi.



Hình 1. Cơ chế Reflected XSS và cách phòng chống bằng mã hoá đầu ra